

New York City Borough Prediction

Aby Joe Jose

1. INTRODUCTION

New York City is a rapidly growing urban center with significant opportunities for business and residential development. This project focuses on building a multiclass classification model to predict the borough of a property in New York City based on features like tax class, sale price, apartment age, unit counts, and square footage. The target classes are Brooklyn, Queens, Bronx, Staten Island, and Manhattan. Given the complexity of NYC's real estate market, the model aims to support stakeholders in making informed and efficient decisions for business or residential planning.

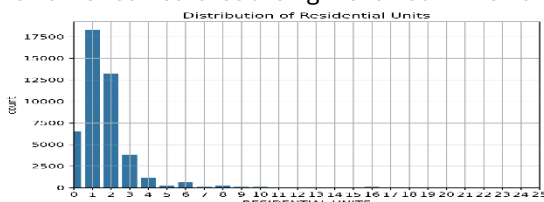
The dataset used in this project is the [NYC Property Sales](#) dataset, publicly available on Kaggle. It contains detailed information on real estate transactions across New York City's five boroughs. The data includes attributes such as sale price, tax class, number of residential and commercial units, building square footage, and more

2. EXPLORATORY DATA ANALYSIS (EDA)

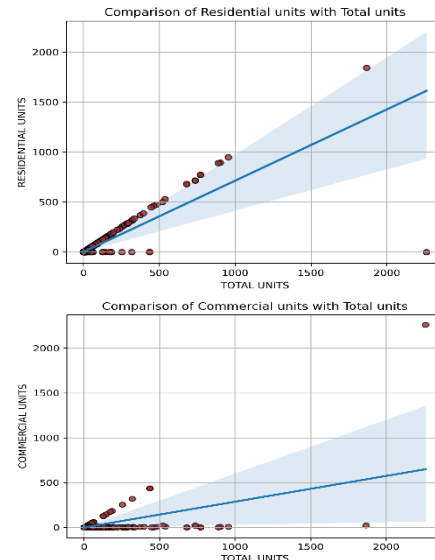
- The dataset has **84548** rows and **22** columns
- Each row is a sale transaction of an apartment in New York city with its demographics
- Some of the columns in the dataset are:

Columns	Description
Tax class	Classification based on the number of units
Tax class at the time of sale	Based on the units at time of sale

- There are 39,215 rows with null values and in those rows most of the variables are missing. There are no duplicate rows.
- The target variable, **Borough**, is imbalanced, with **Brooklyn** representing the largest portion at **48%** of the data. This is followed by **Queens (24%)**, **Bronx (13%)**, and **Staten Island (11%)**, while **Manhattan** accounts for the smallest share with only **2%** of the dataset.
- All the numerical features are right skewed in their distribution



- Residential units have more ratio compared to commercial units with respect to the Total units



3. DATA PREPARATION AND PREPROCESSING

- The columns Year built was converted into building age by subtracting from current year (2025)
- The dataset was split into training, validation, and test sets, with **70%** allocated for training. The remaining **30%** was equally divided between validation and test sets.
- All the missing rows have more than two columns missing so deleted all the missing rows.
- Redundant columns and those with a high number of categories were removed, along with features like address and zip code to prevent potential horizontal **data leakage**.
- No text or time features are included in the dataset for analysis, so no vectorizer is used.
- A data pipeline was implemented for numerical features, where missing values were imputed using the median value, followed by a **log transformation** to reduce skewness, and then scaled using **standard scaler**.
- Data pipeline was created for categorical features to impute any null values with most frequent value, followed by an encoder to **one Hot encode** categorical variables to numeric values

New York City Borough Prediction

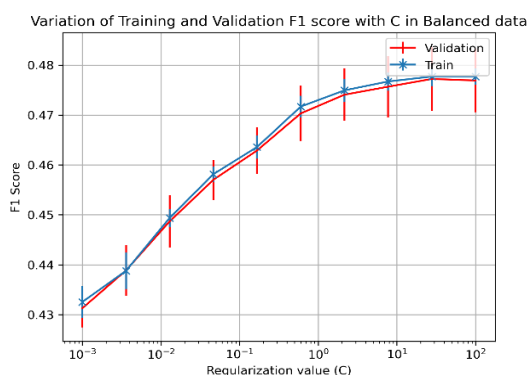
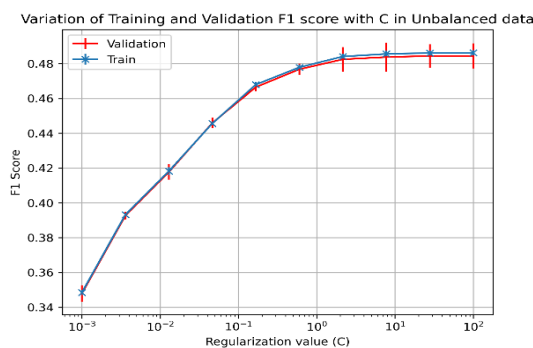
Aby Joe Jose

- The data was highly imbalanced, so created a separate pipeline using **Imblearn** to balance, by down sampling first and then oversampling by SMOTE. After balancing all classes have 2700 rows.
- Balanced and Unbalanced data sets were used for logistic regression model building; all other models were trained with **unbalanced** datasets.

4. Model Training and Results

1. Logistic Regression (Balanced and Unbalanced)

- Both Balanced and Unbalanced datasets were passed to separate logistic regression models in grid search with 5 folds and the parameters given for tuning were **C** with equal spaced ten **log values between -3 and 2**
- Best parameter (C) given by both the models was **27.82**.

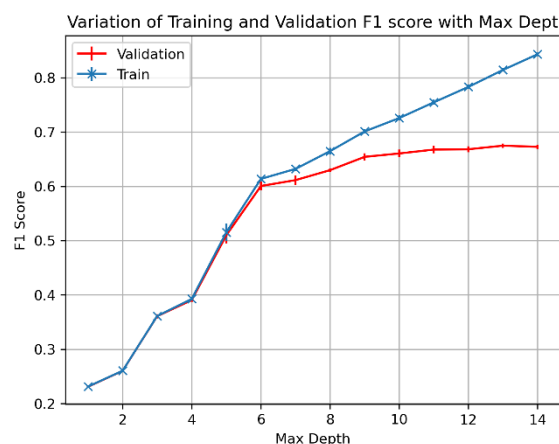


- Unbalanced data set shows lower standard deviation in validation data compared to balanced dataset.
- The results on Test set are as follows: Surprisingly unbalanced dataset model performed better than the model with balanced dataset.

Model	F1 score (macro)	Accuracy(test)
Logistic regression (Balanced)	0.47	0.53
Logistic regression (Unbalanced)	0.48	0.63

2. Decision Tree Classifier

- Unbalanced dataset was used for the model in grid search with 5 folds and parameters passed to it were **max depth** with values from **1 to 15**
- Best parameter for the model had a depth of **13** and a best f1 score of **0.67** on train set.
- The model with best parameters performed nicely on the test dataset with an **accuracy of 0.72** and **f1 (macro) of 0.68**



- The model starts to overfit as max depth increases.

3. SVC

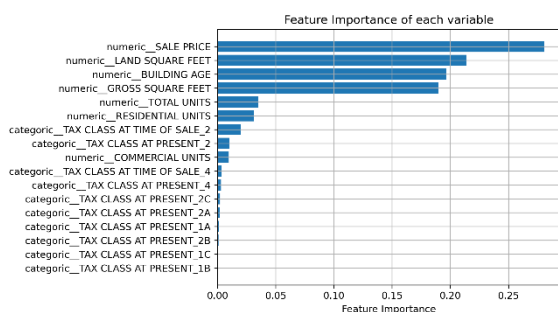
- The data was resampled to 1000 as SVC takes a lot of time to process along with grid search
- Three parameters were given to tune, including **C** with equal spaced 10 log values between **0.1** and **3**, **Gamma** with equal spaced 10 values between **0.1** and **5** and two **Kernels – RBF and Poly**.
- Of the total of 1000 fits the best parameters were **C: 51.41**, **Gamma: 5.0** and **Kernel: RBF**. The best f1 score on the train set was 0.55
- The model with best parameters gave an **F1 score of 0.53** and **accuracy of 0.63** on test dataset.

New York City Borough Prediction

Aby Joe Jose

4. Random Forest

- Unbalanced dataset was used in the model with grid search with a **max depth** ranging from **10 to 30** and **estimators** from **100 to 1000**
- Of the 225 fits the best model gave the best parameter with a max depth of **20** and number of estimators as **700**, and a best score of **0.75** in train dataset.



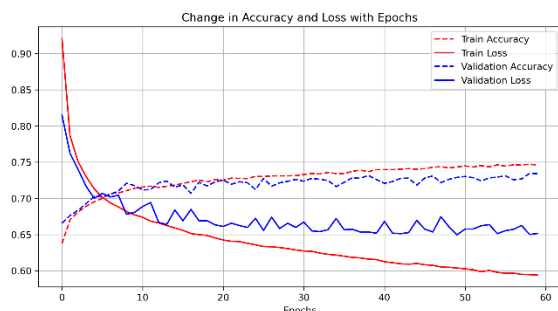
- The model worked well in test dataset with an **accuracy** of **0.76** and a **F1** score of **0.72**
- Sale price and land square feet are contributing the most to the model.

5. Gradient Boosting

- The model was designed with a depth of 4, estimators as 500, and for **early stopping** number of iterations for no change was given as **10**.
- The model had 14 tresses and a best score of **0.71** on train set and f1 macro score of **0.67** and accuracy of **0.73** on test dataset.

6. Neural Networks

- Multiple models were made with different combinations of neurons, layers and dropout layers.
- All models had callbacks set with a patience of 10.
 - Of the 5 models the model with 5 hidden layers showed better results, the model had higher number of



neurons on the initial layers with 100 neurons on the initial layers and 10 neurons on the later layers.

- Decrease in **loss** in validation is not as evident as in training as epochs increase.
- The best model gave the least **loss** of **0.65**, **accuracy** of **0.74** and a f1 score **0.68**.

5. CONCLUSION

Among the models evaluated, simpler models like **Logistic Regression** showed limited performance, with an F1 score of **0.48** on both the training and test sets, and a test accuracy of **63%**. As model complexity increased, performance generally improved. **Support Vector Classifier (SVC)** also struggled to generalize well, yielding a test F1 score of **0.53**. In contrast, tree-based models such as **Decision Tree**, **Random Forest**, and **Gradient Boosting** demonstrated stronger performance, with Random Forest emerging as the best overall model.

Random Forest achieved the highest test F1 macro score of **0.72** and a test accuracy of **76%**, outperforming other models across both metrics. **Gradient Boosting** and **Decision Tree** followed closely with test F1 scores of **0.67** and **0.68**, respectively. The **Neural Network** also performed reasonably well, with a test F1 score of **0.68**, showing potential but not surpassing Random Forest.

Model	Train F1 score	Test F1 Score	Test Accuracy
Logistic (Unbalanced)	0.48	0.48	0.63
Decision Tree	0.67	0.68	0.72
SVC	0.55	0.53	0.63
Random	0.75	0.72	0.76
Gradient	0.71	0.67	0.73
Neural	0.65	0.68	0.72

Although Logistic Regression did not perform well on the balanced dataset, this does not necessarily mean balanced data cannot be handled effectively. More complex models such as **Random Forest** and **Gradient Boosting** showed stronger performance despite the class imbalance. In future work, experimenting with **class balancing techniques** such as SMOTE, may further improve the performance, especially for models sensitive to imbalance.

Overall, the results indicate that ensemble-based models, particularly **Random Forest**, are better suited for this multiclass classification task involving NYC property data. With a test F1 score of **0.72** and accuracy of **76%**, it provides a strong balance between bias and variance, making it the most effective model in this analysis